

Linux OS Mini-Project Proposal

13-Day Build Plan — Scope, Stack, Bottlenecks & Workflow

Prepared for team review and sign-off before development starts.

Excluded: Project 1 — Kernel Sprint (Process Management)

TAG: NOT SELECTED — Reason: Timeline risk too high for 13 days.

This project requires patching real Linux kernel scheduler code (kernel/sched/*.c), rebuilding the kernel, booting it in a VM, and benchmarking with tools like perf, hackbench and the Linux Test Project (LTP) for POSIX compliance. A single kernel rebuild cycle alone can take 20-40 minutes, and the codebase is millions of lines with a genuinely dangerous failure mode (unbootable kernel, corrupted test VM). Realistic academic timelines for this kind of work are measured in months, not two weeks.

- No mature open-source "AI assistant" layer to lean on — everything is raw C systems work.
- High risk of losing days to environment setup (cross-compiling, QEMU/KVM config) before any real feature work starts.
- Benchmarking needs statistical rigor (multiple runs, controlled environment) to produce a credible result — easy to get a misleading number under time pressure.
- Team has no slack in the schedule to recover from a bad kernel build or a botched benchmark run.

Decision: Dropped in favor of Projects 2, 3 and 4, which are userspace applications with mature libraries, safe fail modes (worst case: a bug, not a crashed machine), and a clear demo path within 13 days.

Project 2 — AI-Powered Intelligent Storage Optimizer

Goal: Scan a Linux filesystem, find duplicate and unused files, forecast future storage growth, and recommend safe cleanup/archiving actions — with a lightweight TUI/GUI dashboard.

Tech Stack

- **Go** — filesystem walker + hasher (concurrency via goroutines, single static binary, fast on large trees).
- **Python** — forecasting/AI layer (pandas, scikit-learn / simple linear regression or Prophet-style trend model) and recommendation logic.
- **SQLite** — local index of scanned file metadata (path, size, hash, atime/mtime) so re-scans are incremental, not full re-walks.
- **TUI:** Go's Bubble Tea / tview, or **lightweight GUI:** a small Python (Textual / PySide) or Go+Wails dashboard — pick one, not both.
- Go and Python communicate via a local JSON-RPC/HTTP call or simply: Go writes to SQLite, Python reads from it. No custom protocol needed.

What Already Exists (for reference / inspiration)

- ncdu, dust, duf — fast disk usage visualizers (no duplicate detection, no AI, no forecasting).
- fdupes, rfind, jdupes — pure duplicate-file finders (no unused-file logic, no prediction, no recommendations).
- BleachBit — rule-based cleanup tool (static rules, not adaptive, no forecasting, no natural-language explanation).
- None of these combine duplicate detection + staleness detection + growth forecasting + plain-language recommendations in one tool — that combination is the gap we're filling.

What We Are Building

- A single tool that scans once, builds a local index, and continuously answers: what's duplicated, what's stale, and how fast is disk usage growing.
- A forecasting layer that turns raw growth numbers into an actual warning ("you'll hit 90% capacity in ~18 days at this rate").
- A recommendation layer that explains findings in plain language and only acts after explicit user confirmation — never auto-deletes.

Key Bottlenecks & How We Solve Them

Bottleneck	Why It Happens	Our Fix
Slow full-tree scan	Millions of stat()/readdir() calls on large home dirs or NAS mounts	Go goroutine worker pool walking directories concurrently; incremental re-scans using stored mtimes
Hashing every file is wasteful	Hashing large files just to compare them is CPU/I-O heavy	Two-pass strategy: group by file size first (cheap), hash only files that share a size
False positives on "unused" files	atime is unreliable on many mounts (relatime/noatime), simple thresholds mislabel critical files	Weight staleness by file type/path (e.g. never flag dotfiles/config as unused) and always require confirmation
Naive Python scan is slow	Per-file interpreter overhead across millions of iterations	Push the hot loop (walk + hash) into Go; Python only touches the already-aggregated SQLite data

Development Workflow (13 Days)

- Day 1-2: Go scanner — concurrent directory walk, metadata capture, SQLite schema + writer.
- Day 3-4: Duplicate detection — size-bucket grouping, xxHash/SHA-256 hashing, dedup report.
- Day 5-6: Unused-file heuristics — atime/mtime scoring, path/type weighting rules.
- Day 7-8: Python forecasting module — read history snapshots from SQLite, fit growth trend, project future usage.
- Day 9-10: Recommendation layer — plain-language summaries, optional LLM call for explanations, confirm-before-action logic.
- Day 11-12: TUI/GUI dashboard wiring — display scan results, duplicates, forecast chart, action buttons.
- Day 13: End-to-end testing on a real home directory, polish, README + demo script.

Runtime Workflow (How the Finished Tool Works for a User)

- User runs the scanner (e.g. **storage-optimizer scan ~/**) — Go walks the tree and populates SQLite.
- Dashboard opens showing: total space, duplicate groups, stale files, and a forecast graph.
- User reviews flagged items with plain-language explanations ("4.2GB in duplicate videos in Downloads").
- User approves specific actions (archive to compressed folder, or delete) — nothing happens automatically.
- Tool executes only the approved actions and updates the index.

Project 3 — AI-Powered Linux Operations Assistant (Natural Language)

Goal: Let a user ask plain-English questions about their Linux system ("why is my disk full", "why is this process using so much RAM") and get a diagnosis, relevant files/docs, and recommended commands.

Tech Stack

- **Python** end-to-end — this is an orchestration problem, not a performance problem.

- LLM integration via Anthropic/OpenAI SDK (or a local model through Ollama for an offline mode).
- **psutil** for structured system stats (CPU, memory, processes) instead of parsing raw CLI text where possible.
- **subprocess** to call `df`, `free`, `journalctl`, `dmesg`, `ps` for ground-truth diagnostics the LLM can reason over.
- Semantic file/doc search: sentence-transformers embeddings + FAISS or ChromaDB as a local vector store.
- Interface: Textual (Python TUI) for a terminal chat-style UI, or a minimal web UI (FastAPI + simple HTML) if a browser view is preferred.

What Already Exists

- ShellGPT, Warp AI, GitHub Copilot CLI — natural-language-to-command tools (mostly command generation, limited live diagnostics gathering).
- `htop`, `glances` — great live system monitors, but no natural language layer and no explanation of findings.
- None of these combine live diagnostic data collection + semantic document search + plain-language root-cause explanation in one flow — that's the gap.

What We Are Building

- A query router that decides which diagnostics to pull for a given question (disk, memory, CPU, or logs).
- A diagnostics executor that gathers real, structured system data before the LLM ever answers — grounding responses in facts, not guesses.
- A semantic search layer over local files/configs so questions like "where did I set my hostname" resolve to an actual file.
- A response layer that explains the root cause in plain language and lists recommended commands, without auto-running anything risky.

Key Bottlenecks & How We Solve Them

Bottleneck	Why It Happens	Our Fix
LLM hallucinating system state	Model has no live view of the actual machine	Always run real diagnostics first (<code>df/free/ps/journalctl</code> via <code>psutil+subprocess</code>) and feed structured output into the prompt as grounding context
Ambiguous queries	"My system is slow" could mean CPU, memory, disk, or I/O	Router step classifies intent first, pulls only the relevant diagnostic set, asks one clarifying question only if truly ambiguous
Parsing inconsistent CLI output	Text output of <code>df/free/ps</code> varies by <code>distro/locale</code>	Prefer <code>psutil</code> (structured Python objects) over raw text parsing wherever possible; regex only as fallback
Slow/irrelevant file search	Plain <code>grep</code> over large trees is slow and keyword-only	Precomputed embedding index (FAISS/ChromaDB) for semantic search, refreshed incrementally

Development Workflow (13 Days)

- Day 1-2: Project architecture + LLM SDK integration + basic query-response loop.
- Day 3-5: Diagnostics executor — `psutil/subprocess` wrappers for disk, memory, CPU, and log analysis; structured output format.
- Day 6-8: Semantic file/doc search — embedding index build + incremental refresh + query interface.
- Day 9-11: Response synthesis — prompt design that grounds answers in diagnostic data, command recommendation formatting.

- Day 12-13: Safety guardrails (never auto-execute), Textual/web UI polish, end-to-end testing across a few real scenarios (disk full, high memory, error logs).

Runtime Workflow (How the Finished Tool Works for a User)

- User types a question in the assistant's chat UI (e.g. "why is my disk full?").
- Router decides this needs disk diagnostics — assistant runs `df/du/journalctl` locally.
- Structured findings + the user's question go to the LLM as grounded context.
- Assistant returns a plain-language diagnosis and a short list of recommended commands.
- User reviews and manually runs the suggested commands, or confirms a one-click "run this for me" step.

Project 4 — AI-Based System Intent Engine for Safe Command Execution

Goal: Intercept commands before they run, detect risky operations, explain the impact in plain language, and suggest safer alternatives — without slowing down normal command use.

Tech Stack

- **Bash/Zsh hook** (`zsh preexec`, or `bash DEBUG` trap) — this is the interception layer; not really a language choice, it is required shell integration.
- **Python background daemon** (kept warm via a local socket or a small FastAPI server) — holds the rule engine and calls the LLM, so we avoid interpreter start-up lag (~30-100ms) on every single command.
- Optional upgrade if time allows: rewrite just the fast-path rule engine in **Go** as a tiny always-on binary for near-zero latency; LLM reasoning stays in Python either way.
- LLM call only for ambiguous cases the rule engine doesn't resolve — keeps the common case fast.

What Already Exists

- `safe-rm`, `rm -i` aliases — basic guardrails around single commands, no intent reasoning, no explanation.
- `trash-cli` — redirects deletes to a trash folder, but doesn't reason about risk or explain impact.
- `sudo` with command whitelisting (`visudo` rules) — access control, not intent understanding.
- None of these explain **why** a command is risky in plain language or propose a corrected alternative before execution — that's the gap this project fills.

What We Are Building

- A transparent safety layer that sits between typing a command and it actually running.
- A fast rule-based classifier for well-known dangerous patterns (`rm -rf /`, `dd` to a raw device, `chmod -R 777 /`, `mkfs`, fork bombs, `curl | sh`).
- An LLM-based reasoning fallback for commands that aren't obviously dangerous but look like a mistake (e.g. a glob pattern that matches more than intended).
- A confirm/abort/edit prompt so the user stays in control — nothing is ever blocked silently or auto-corrected without consent.

Key Bottlenecks & How We Solve Them

Bottleneck	Why It Happens	Our Fix
Added latency on every command	Spawning a fresh Python process per command adds noticeable lag	Run the engine as a warm background daemon; shell hook is a thin client over a local socket/HTTP call

Over-flagging safe commands	Overly broad regex rules trigger on legitimate commands, annoying the user	Curated, tested pattern library scoped to genuinely destructive operations; LLM only used for the ambiguous middle ground
Missing novel/creative dangerous commands	Rules can't cover every destructive pattern in advance	LLM reasoning fallback evaluates intent/impact for anything the rule engine doesn't confidently classify as safe
Shell edge cases (aliases, pipes, sudo, subshells)	Naive parsing misses the real command being run	Test suite of dangerous-command variants covering pipes/sudo/subshells built specifically to catch these before launch

Development Workflow (13 Days)

- Day 1-2: Shell hook working reliably (zsh preexec / bash DEBUG trap) + basic pass-through to a local daemon.
- Day 3-5: Rule-based risk engine — build and test the dangerous-pattern library.
- Day 6-8: LLM-based intent reasoning for ambiguous cases not caught by rules.
- Day 9-10: Safer-alternative suggestion logic (e.g. propose find -mtime +30 -delete instead of a broad rm -rf).
- Day 11-12: Confirmation UX (confirm/abort/edit) + handling aliases, pipes, sudo, subshells.
- Day 13: Run against a dangerous-command test suite, polish, demo script.

Runtime Workflow (How the Finished Tool Works for a User)

- User types a command in their normal terminal, e.g. rm -rf /var/log/*.
- Shell hook intercepts it before execution and sends it to the local daemon.
- Rule engine flags it as risky (deletes active log files) — instantly, no LLM needed for this obvious case.
- Engine explains the impact in plain language and suggests a safer alternative command.
- User types y to proceed, n to abort, or edits the command — only then does anything actually execute.

Difficulty & Scope Comparison

	Project 1 (Kernel)	Project 2 (Storage)	Project 3 (NLQ Assistant)	Project 4 (Intent Engine)
Kernel-level work	Yes (real)	No	No	No
Core difficulty	Very High	Medium	Medium-High	Medium
Crash/instability risk	High (VM only)	None	Low	Low
Primary stack	C (kernel)	Go + Python	Python	Bash hook + Python (+ optional Go)
AI/LLM depth needed	None	Light (forecasting)	Heavy (orchestration)	Medium (rules + LLM fallback)
13-day feasibility	Narrow slice only	Full MVP realistic	Full MVP, scoped diagnostics	Full MVP realistic
Demo impact	Low (needs graphs)	Medium	High	Very High
Selected for build?	No	Yes	Yes	Yes

Summary: Projects 2, 3, and 4 are all userspace applications with mature libraries, safe failure modes, and a realistic path to a working demo inside 13 days. Project 1 is excluded purely on timeline risk — the kernel build/boot/benchmark loop and POSIX-compliance validation needed for a credible result do not fit a two-week

window without a serious chance of losing days to environment or stability issues.